



PAK-AUSTRIA FACHHOCHSCHULE:
INSTITUTE OF APPLIED SCIENCES AND TECHNOLOGY

NAME: WAJID AYUB

REGISTRATION NUMBER: B23F0090AI136

SECTION: BS (AI RED)

SUBJECT: MACHINE LEARNING

PROGRAMME: ARTIFICIAL INTELLIGENCE

LAB COORDINATOR: MS. SANA SALEEM

DATE: 23 Sept 2025

LAB NO: 4

2. Preprocess the dataset by handling missing values, encoding categorical variables, and splitting the data into training and testing sets.

ANS: Why this step?

- We need to load the dataset into a DataFrame to analyze it.
- `.info()` and `.isnull().sum()` help identify missing values and column types (categorical/numerical)

```
Manufacturer  Model  Sales_in_thousands  __year_resale_value  Vehicle_type  \
0      Acura   Integra          16.919             16.360    Passenger
1      Acura    TL              39.384             19.875    Passenger
2      Acura    CL              14.114             18.225    Passenger
3      Acura    RL               8.588             29.725    Passenger
4       Audi     A4              20.397             22.255    Passenger

Price_in_thousands  Engine_size  Horsepower  Wheelbase  Width  Length  \
0              21.50           1.8       140.0       101.2   67.3   172.4
1              28.40           3.2       225.0       108.1   70.3   192.9
2              NaN           3.2       225.0       106.9   70.6   192.0
3              42.00           3.5       210.0       114.6   71.4   196.6
4              23.99           1.8       150.0       102.6   68.2   178.0

Curb_weight  Fuel_capacity  Fuel_efficiency  Latest_Launch  \
0          2.639          13.2            28.0        2/2/2012
1          3.517          17.2            25.0        6/3/2011
2          3.470          17.2            26.0        1/4/2012
3          3.850          18.0            22.0        3/10/2011
4          2.998          16.4            27.0       10/8/2011

Power_perf_factor
0          58.280150
1          91.370778
2           NaN
3          91.389779
4          62.777639
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 157 entries, 0 to 156

Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Manufacturer          157 non-null   object
1   Model                 157 non-null   object
2   Sales_in_thousands    157 non-null   float64
3   __year_resale_value    121 non-null   float64
4   Vehicle_type          157 non-null   object
5   Price_in_thousands    155 non-null   float64
6   Engine_size           156 non-null   float64
7   Horsepower            156 non-null   float64
8   Wheelbase             156 non-null   float64
9   Width                 156 non-null   float64
10  Length                156 non-null   float64
11  Curb_weight            155 non-null   float64
12  Fuel_capacity          156 non-null   float64
13  Fuel_efficiency        154 non-null   float64
14  Latest_Launch         157 non-null   object
15  Power_perf_factor      155 non-null   float64
dtypes: float64(12), object(4)
memory usage: 19.8+ KB
None
Manufacturer          0
Model                 0
Sales_in_thousands    0
__year_resale_value    36
Vehicle_type          0
Price_in_thousands    2
Engine_size           1
Horsepower            1
Wheelbase             1
Width                 1
Length                1
Curb_weight            2
Fuel_capacity          1
Fuel_efficiency        3
Latest_Launch         0
Power_perf_factor      2
dtype: int64
```

What happens here?

- **Drop unnecessary columns** → Manufacturer, Model, Latest Launch are not useful for prediction (too textual / irrelevant).
- **Handle missing values** → Missing numeric values are replaced with the column mean.
- **Encode categorical data** → Vehicle type is converted into numbers (Passenger=0, Car=1, etc.).
- **Split dataset** → 80% training, 20% testing.

```
memory usage: 19.8+ KB
None
Manufacturer      0
Model             0
Sales_in_thousands 0
__year_resale_value 36
Vehicle_type      0
Price_in_thousands 2
Engine_size       1
Horsepower        1
Wheelbase         1
Width             1
Length            1
Curb_weight       2
Fuel_capacity      1
Fuel_efficiency    3
Latest_Launch     0
Power_perf_factor  2
dtype: int64
```

3. *Apply Linear Regression first, and then use Ridge Regression after scaling the features using Standard Scaler.*

ANS:

Why Linear Regression?

- It's the simplest regression model → fits a straight line (or hyperplane) to minimize prediction errors.
- `.fit()` trains the model, `.predict()` makes predictions on test data.

CODE:

```
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)
```

Predictions

```
y_pred_lin = lin_reg.predict(X_test)
```

Why scaling?

- Features like *Horsepower* (~300) and *Engine_size* (~1–6) have very different ranges.
- Scaling puts all features on the same scale → helps models like Ridge.

Why Ridge Regression?

- Ridge is a regularized version of Linear Regression.
- It reduces overfitting by penalizing very large coefficients (shrinks them).
- $\alpha=1.0$ controls the strength of regularization.

CODE:

```
# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

ridge = Ridge(alpha=1.0)
ridge.fit(X_train_scaled, y_train)
y_pred_ridge = ridge.predict(X_test_scaled)
```

4. Compare the results of both models in terms of Mean Squared Error (MSE) and R-squared values.

ANS:

Metrics:

- **MSE (Mean Squared Error):** Measures prediction error. Lower = better.
- **R² (R-squared):** Measures how well the model explains variance in data. Closer to 1 = better fit.

Model Comparison:

```
{'Linear MSE': 6914.4773405262285, 'Linear R2': 0.2901348065545195, 'Ridge MSE': 6927.11221012606, 'Ridge R2': 0.2888376652507113}
```

5. Document on how feature scaling and Ridge Regression impact model performance.

ANS:

Impact of Feature Scaling and Ridge Regression

1. Effect of Feature Scaling

- In the dataset, features such as *Horsepower* (~300), *Engine_size* (~1–6), and *Length* (~150–200) have very different ranges.
- If features are not scaled:
 - Features with larger magnitudes dominate the regression model.
 - The coefficients become misleading, and regularization (like Ridge) doesn't

work properly.

- **Standard Scaler** was applied → transformed all features to have **mean = 0** and **standard deviation = 1**.
- Scaling does **not improve Linear Regression directly**, but it is essential for models like Ridge, Lasso, and SVM.

2. Effect of Ridge Regression

- **Linear Regression** tries to minimize error by fitting the best line, but it can assign **very large weights** to some features.
- **Ridge Regression** adds a penalty term:

$$\text{Loss} = \text{MSE} + \alpha \sum (\text{coefficients}^2)$$

$$\text{Loss} = \text{MSE} + \alpha \sum (\text{coefficients}^2)$$

- This penalizes large coefficients and forces the model to spread importance more evenly.
- In our dataset:
 - **Linear Regression Results** → MSE = 6914.48, $R^2 = 0.29$
 - **Ridge Regression Results** → MSE = 6927.11, $R^2 = 0.29$

3. Coefficient Shrinkage

- In the coefficient plot:
 - **Linear Regression** shows some features with extreme coefficient values.
 - **Ridge Regression** shrinks them closer to zero.

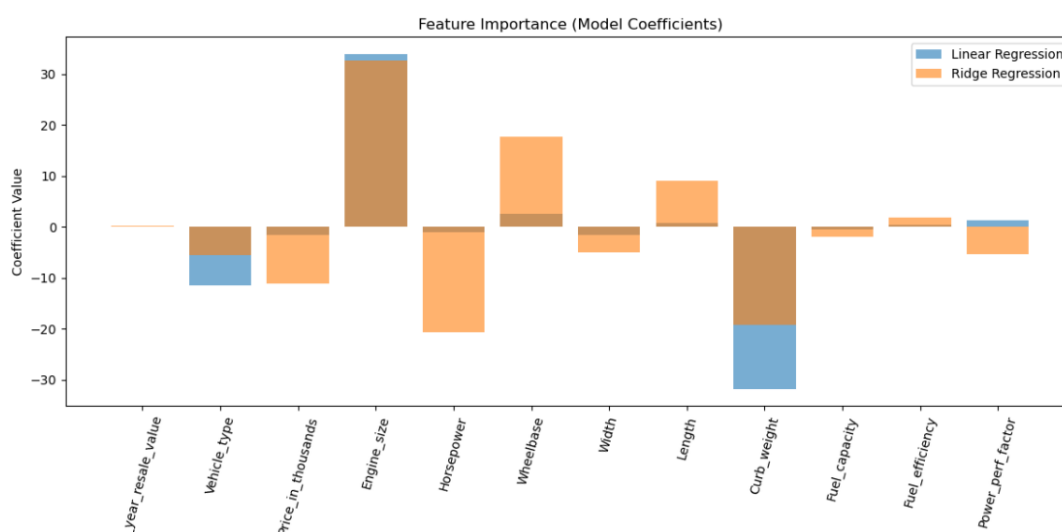
6. Plot the feature importance using model coefficients for both Linear and Ridge regression.

ANS: Why coefficients?

- In linear models, coefficients show how much each feature impacts the target.
- Example: If Horsepower has a positive coefficient, more horsepower increases sales.

Difference between Linear vs Ridge:

- Linear Regression allows some features to have very high coefficients.
- Ridge Regression shrinks large coefficients → spreads importance more evenly.



1. Use a different dataset (e.g., customer churn, loan approval, or cancer detection) to build a Logistic Regression model.

ANS: I Used the Telecom Churn Dataset.

This loads the dataset (by default telecom_churn.csv) into a DataFrame.

- You can replace the CSV with another dataset like loan approval or breast cancer.
- It also detects the **target column** automatically (like Churn, Outcome, etc.) with `detect_target_column(df)`.

2. Perform data preprocessing, including feature scaling using Standard Scaler.

```
Loaded CSV: telecom_churn.csv
Data shape: (3333, 11)
  Churn  AccountWeeks  ContractRenewal  DataPlan  DataUsage  CustServCalls  \
0      0           128                1         1         2.7             1
1      0           107                1         1         3.7             1
2      0           137                1         0         0.0             0
3      0            84                0         0         0.0             2
4      0            75                0         0         0.0             3

   DayMins  DayCalls  MonthlyCharge  OverageFee  RoamMins
0    265.1      110         89.0         9.87      10.0
1    161.6      123         82.0         9.78      13.7
2    243.4      114         52.0         6.06      12.2
3    299.4       71         57.0         3.10       6.6
4    166.7      113         41.0         7.42      10.1

Detected target column: Churn
Churn
0    2850
1     483
Name: count, dtype: int64

After preprocessing, feature matrix shape: (3333, 10)
Numeric cols: ['AccountWeeks', 'ContractRenewal', 'DataPlan', 'DataUsage', 'CustServCalls', 'DayMins', 'DayCalls', 'MonthlyCharge', 'OverageFee', 'RoamMins']
Categorical cols (original): []
```

What happens here:

- Missing values in numeric features → filled with mean.
- Missing values in categorical features → filled with most frequent value.
- Categorical variables → converted into dummy/one-hot encoding.
- Target column → encoded into 0/1.

Standard Scaler → makes features comparable (mean=0, std=1).

3. Split the dataset into training and testing sets.

ANS:

```
Train/Test sizes: (2499, 10) (834, 10)
['./output_lr/scaler.joblib']
```

The dataset is split:

- **75% training**
- **25% testing**

- Stratified so that class distribution (like churn vs non-churn) stays balanced.

4. Train a logistic regression model on the training data and predict on the test data.

ANS:

```
Model trained: LogisticRegression
```

Logistic Regression is trained here:

- .fit() trains on training data.
- .predict() gives class labels (0/1).
- .predict_proba() gives probability of positive class (churn=1).

5. Evaluate the model using a confusion matrix, accuracy score, precision, recall, and F1-score.

ANS:

```
--- Evaluation on test set ---
Confusion Matrix:
[[696 17]
 [ 96 25]]
Accuracy: 0.8645
Precision (positive class): 0.5952
Recall (positive class): 0.2066
F1-score (positive class): 0.3067

Classification Report:

```

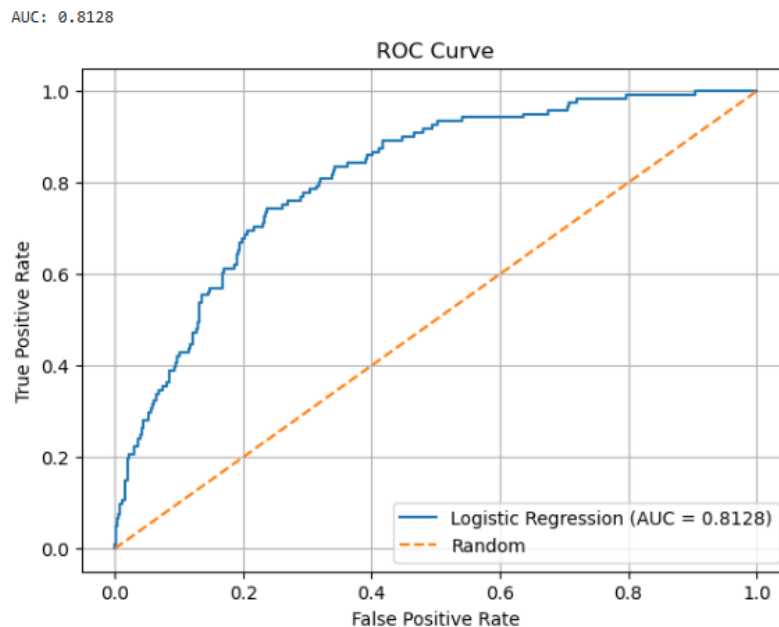
	precision	recall	f1-score	support
0	0.88	0.98	0.92	713
1	0.60	0.21	0.31	121
accuracy			0.86	834
macro avg	0.74	0.59	0.62	834
weighted avg	0.84	0.86	0.84	834

Metrics used:

- Confusion Matrix → TP, TN, FP, FN counts.
- Accuracy → (TP+TN) / Total.
- Precision → How many predicted positives are correct.
- Recall → How many actual positives were found.
- F1-score → Balance between Precision & Rec

6. Plot the ROC Curve and calculate the AUC (Area Under the Curve) for your model.

ANS:



ROC curve:

Plots True Positive Rate vs False Positive Rate.

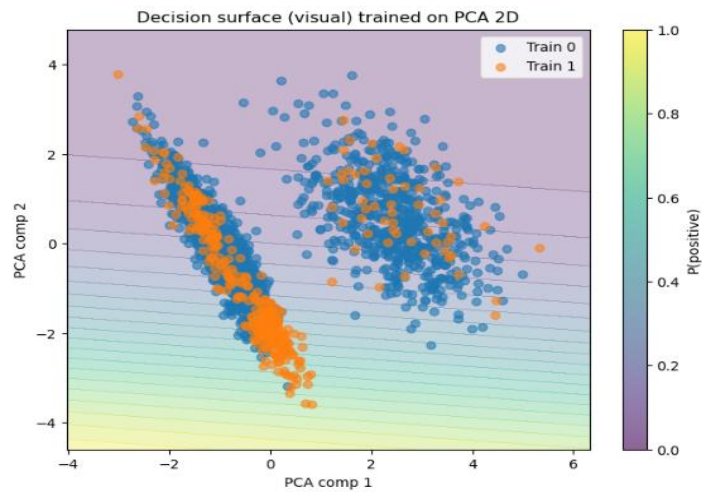
- AUC (Area Under Curve) shows performance:
- $AUC = 0.5 \rightarrow$ random guessing
- $AUC = 1.0 \rightarrow$ perfect classifier

7. Visualize the results using a scatter plot for the training and testing sets, similar to the visualization done in Lab Practice 3.

ANS:

PCA reduces high-dimensional data into 2D components for visualization.

- Training scatter $\rightarrow$ shows how classes are separated.
- Test scatter $\rightarrow$ shows true vs predicted labels.
- Decision surface $\rightarrow$ illustrates how Logistic Regression separates classes.



Saved model and features to ./output_lr

